CS-2710-1 Spring 2026

Computer Organization and Systems (COS)

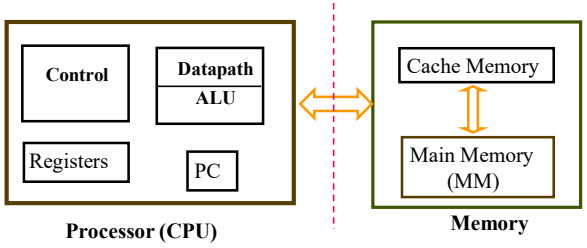
Lecture #20

17 April 2026

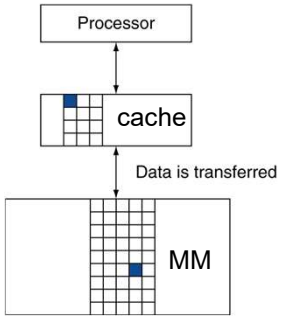
Ashoka University

BBB

CPU ⇔ Cache ⇔ MM



Memory Hierarchy Levels



- Block (*aka* line): unit of copying
  - May be multiple words
- If requested item (I/D) is present in upper level
  - Hit: access satisfied by upper level
    - Hit ratio: hits/accesses
- If requested item is absent
  - Miss: block copied from lower level
    - Time taken: miss penalty
    - Miss ratio: misses/accesses = 1 – hit ratio
  - Then the item is supplied from upper level

Issues in Cache Management

- Block placement: MM → Cache
  - direct-mapped, set-associative, fully associative
- Block identification: index, tag, valid-bit
- Block replacement: LRU, LFU, random
- Write strategy (CPU wants to store data in Mem): write-through, write-back (dirty-bit)
- Miss handling:
  - -- cache read miss
  - --- cache write miss
    - * write-allocate, no-write-allocate

Memory Hierarchy Basics

- When a word is not found in the cache, a *miss* occurs:
  - Fetch word from lower level in hierarchy, requiring a higher latency reference
  - Lower level may be another cache or the main memory
  - Also fetch the other words contained within the *block*
    - Takes advantage of spatial locality
  - Place block into cache in any location within its *set*, determined by address

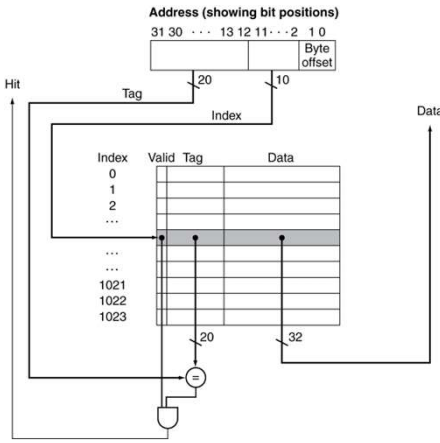
Memory Hierarchy Basics

- Hit-time (time to match a requested item in cache and if found, to transfer it to CPU); time measured in clock cycles;
- Miss rate
  - Fraction of cache requests that result in misses
- Miss-penalty (time to pull an item to cache from the lower level and to transfer it to CPU)
- Causes of misses
  - Compulsory
    - First reference to a block
  - Capacity
    - Blocks discarded and later retrieved
  - Conflict
    - Program makes repeated references to multiple addresses from different blocks that map to the same location in the cache

MM-Address to Cache-Address Resolution (Block size = 1 word)

Hardware realization

MM-address – 32 bits;
Cache: 1 K blocks;
1 block = 1 word
Direct-mapped, write through;
What will be the cache size?

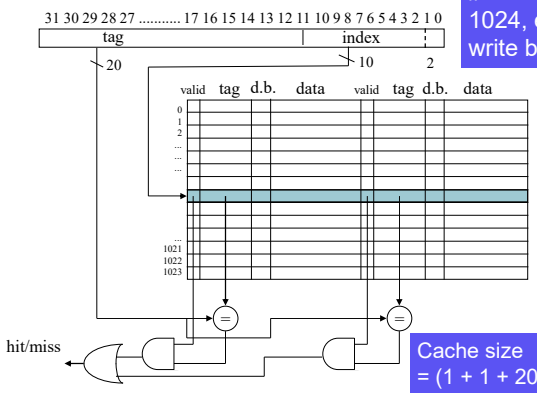


rows (cache-lines) = 1024;
Each row comprises
1 (valid-bit) + 20 (tag-bits) + 32 (data-bits) = 53 bits;
Write-through does not need any extra bit;
Total size of the cache
= 53 ×1024 bits = 53 Kbits

Example 2: Set-Associative Cache

Cache index = MM-address *mod* 1024

MM-address – 32 bits;
Cache: 2 K blocks;
1 block = 1 word (32 bits)
2-way set-associative;
sets (# cache rows) = 1024, each set size = 2;
write back



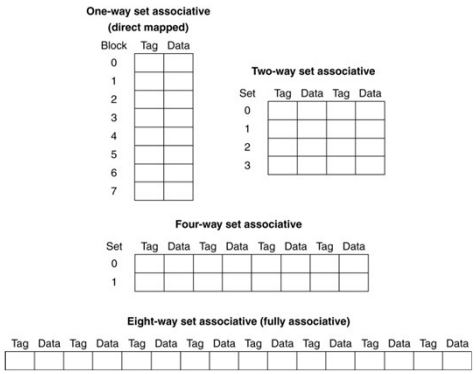
d.b. ⇒ (dirty-bit)

Q. What will happen if this cache becomes fully associative?

Cache size
= (1 + 1 + 20 + 32) × 2 × 1024 bits
= 108 K-bits

Spectrum of Associativity

- For a cache with 8 entries



How to take care of Spatial Locality?

Idea: When an item x (I/D) is requested by CPU:

1. Transfer x from MM → Cache → CPU;

2. Also transfer a few words from MM → Cache that are adjacent to x (even though they have not been requested by CPU, now);

In other words, cache should support the transfer of multiple words from MM;

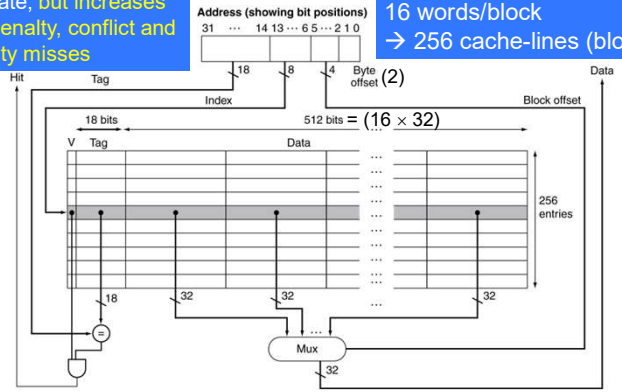
Multi-Word Cache Block

Cache size (for write-through)
= [(1 + 18) + (16 × 32)] × 256 bits
= 132.75 K-bits

Read-request for one word → the full block (16 words) is lifted to cache

Larger cache-blocks aid spatial locality, → reduce compulsory misses and miss-rate; but increases miss penalty, conflict and capacity misses

4 K Cache words;
Multi-word cache-block
16 words/block
→ 256 cache-lines (blocks)



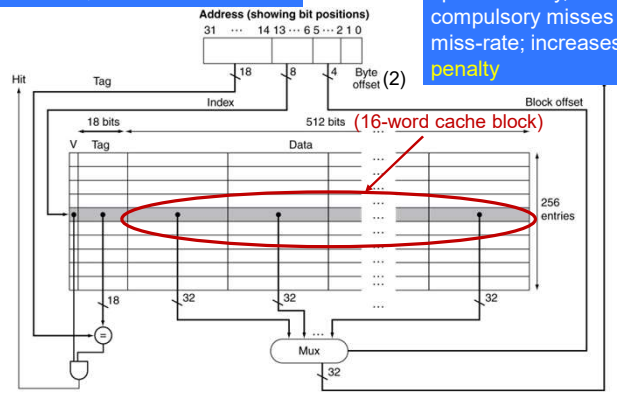
Cache-size for write-back?

Multi-Word Cache Block

Solution: To reduce miss-penalty:
1. Early restart the CPU as soon as the requested word arrives at the cache;
2. Lift the critical-word first, from Mem to cache;

Read-request for one word → the full block (16 words) is lifted to cache

Larger cache-blocks aid spatial locality, → reduce compulsory misses and miss-rate; increases miss penalty



Parameters in Cache Management

Average memory access time (AMAT)
= hit-time + (miss-rate × miss-penalty)
Note: hit-time and miss-penalty are expressed in # clock cycles

Average # memory-stall cycles (AMSC) =
(# memory requests/IC) × miss-rate × miss-penalty

CPU-time = IC × CPI × CCT
= IC × (Ideal-CPI + AMSC) × CCT

Ideal-CPI refers to the case when we have perfect cache, i.e., without any misses

Improving Cache Performance

- Six basic cache optimizations:
  - Larger block size
    - Reduces compulsory misses
    - Increases capacity and conflict misses, increases miss penalty
  - Larger total cache capacity to reduce miss rate
    - Increases hit time, increases power consumption
  - Higher associativity
    - Reduces conflict misses
    - Increases hit time, increases power consumption
  - Higher number of cache levels
    - Reduces overall memory access time
  - Software techniques (programmer or compiler driven)
    (Loop-merging, loop-interchange, cache-friendly data structure)

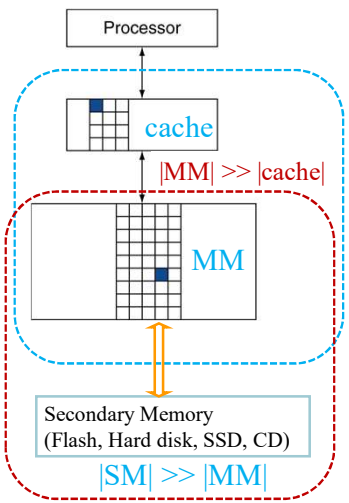
Improving Cache Performance

Example: loop-interchange

Depending on the data structure and/or how the data is stored in MM, cache miss-rate can be reduced just by interchanging the indices in a nested loop.

Virtual Memory

(CPU views SM as if it is MM)



- View main memory (MM) as a
- "cache" for secondary memory (SM)
  - Managed jointly by CPU hardware and the operating system (OS)

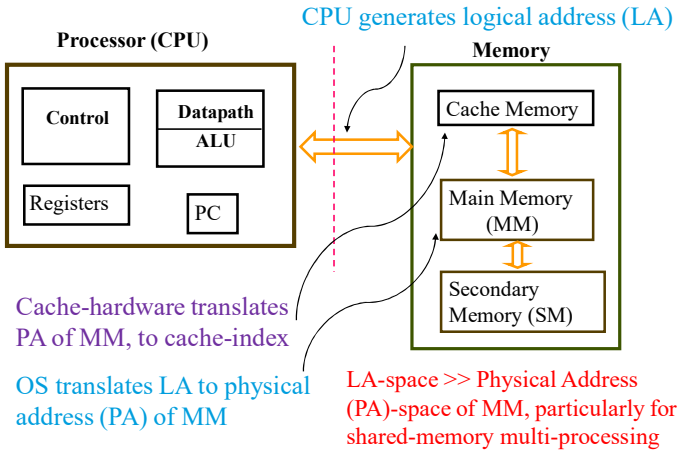
Virtual Memory

- Use main memory as a "cache" for secondary (disk) storage
  - Managed jointly by CPU hardware and the operating system (OS)
- Programs share main memory
  - Each gets a private virtual address space holding its frequently used code and data
  - Protected from other programs
- CPU and OS translate virtual addresses to physical addresses
  - VM "block" is called a page
  - VM translation "miss" is called a page fault

Logical *vs.* Physical Address Space

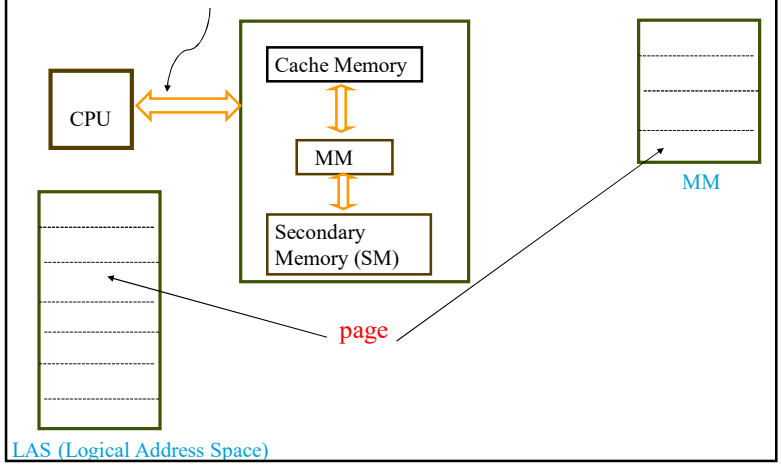
- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as **virtual address**
  - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program

CPU ⇔ Cache ⇔ MM ⇔ SM



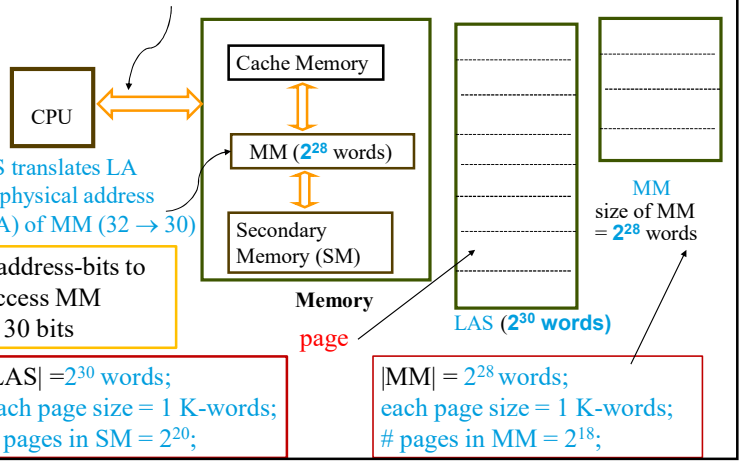
Virtual Memory

CPU generates logical address (LA): 32-bit; can address 2^{30} words;



Virtual Memory

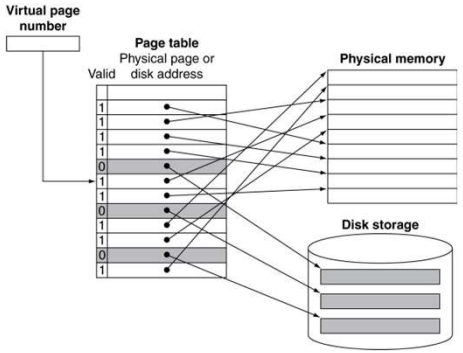
CPU generates logical address (LA): 32-bit; can address 2^{30} words;



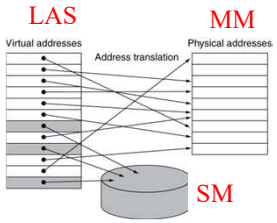
|LAS| = 2^{30} words;
each page size = 1 K-words;
# pages in SM = 2^{20};

|MM| = 2^{28} words;
each page size = 1 K-words;
# pages in MM = 2^{18};

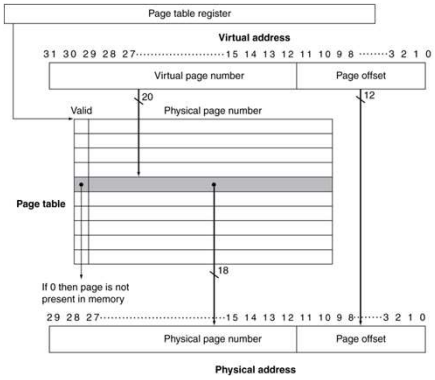
Mapping Pages to Storage

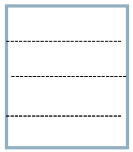


Translation Using a Page Table



Page-size: 4 K-bytes = 1 K-words)





MM
size of MM = 2^{28} words = 2^{30} bytes

LAS (logical address space) = 2^{30} words → 2^{20} pages;

MM (physical address space): = 2^{28} words → 2^{18} pages;

rows in the Page Table = 2^{20};

Page Tables

- Stores placement information
  - Array of page table entries, indexed by virtual page number
  - Page table register in CPU points to page table in physical memory
- If page is present in memory
  - PTE (Page Table Entry) stores the physical page number
  - and other status bits (referenced, dirty, …)
- If page is not present
  - PTE can refer to location in on disk

Page Fault Handler

- Use faulting virtual address to find PTE
- Locate page on disk
- Choose page to replace
  - If dirty, write to disk first
- Read page into memory and update page table
- Make process runnable again
  - Restart from faulting instruction

Page Fault Penalty

- On page fault, the page must be fetched from disk
  - Takes millions of clock cycles
  - Handled by OS code
- Try to minimize page fault rate
  - Fully associative placement
  - Smart replacement algorithms

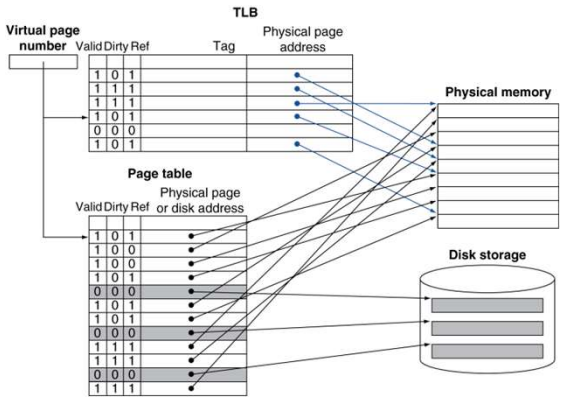
Replacement and Writes

- To reduce page fault rate, prefer least-recently used (LRU) replacement
  - Reference bit (aka use bit) in PTE set to 1 on access to page
  - Periodically cleared to 0 by OS
  - A page with reference bit = 0 has not been used recently
- Disk writes take many cycles
  - Block at once, not individual locations
  - Write through is impractical
  - Use write-back
  - Dirty bit in PTE set when page is written

Fast Translation Using a TLB

- Address translation would appear to require extra memory references
  - One to access the PTE
  - Then the actual memory access
- But access to page tables has good locality
  - So, use a fast cache of PTEs within the CPU
  - Called a Translation Look-aside Buffer (TLB), which is some kind of *cache* for page table
  - Misses could be handled by hardware or software

Fast Translation Using a TLB



TLB Misses

- If page is in memory
  - Load the PTE from memory and retry
  - Could be handled in hardware
    - Can get complex for more complicated page table structures
  - Or in software
    - Raise a special exception, with optimized handler
- If page is not in memory (page fault)
  - OS handles fetching the page and updating the page table
  - Then restart the faulting instruction

Concluding Remarks

- Fast memories are small, large memories are slow
  - We really want fast, large memories ☹
  - Caching gives this illusion ☺
- Principle of locality
  - Programs use a small part of their memory space frequently
- Memory hierarchy
  - L1 cache ↔ L2 cache ↔ … ↔ DRAM memory ↔ disk
- Memory system design is critical for multiprocessors

Important Announcement

Wednesday, 22 April 2026:

Review and Doubt-Clearing Class

COS Class-Test #2

- Friday, 24 April 2026; in class;
  Open-book, open-notes; no electronics;
  Only handwritten sheets, printed material, and books allowed;
  Calculators allowed;
  Credit = 20%;
  Time: 10:15 AM – 11:45 AM;
- Coverage: Lecture #10 - Lecture #20;